

软件工程与计算 III | Ragas 源码泛读报告

华振翔 王涵 柯羽桐 蔡宇阳 张屹峰 张家浩

2026 年，春季学期 | 2026 年 3 月 17 日

目录

- 一、 项目功能描述 3
 - 1. 管理评估样本数据 3
 - 1.1. 创建与管理评估数据 3
 - 1.2. 数据集导出与集成 3
 - 2. 评估指标体系 3
 - 2.1. 丰富的预置指标 3
 - 2.2. 自定义指标能力 3
 - 3. 模型评估执行 3
 - 3.1. 同步与异步评估 3
 - 3.2. 多维度评估报告 3
 - 3.3. 灵活的 LLM 配置 3
 - 4. 实验管理与版本控制 4
 - 4.1. 实验追踪 4
 - 4.2. 版本控制集成 4
 - 4.3. 实验结果存储 4
 - 5. 测试数据自动生成 4
 - 5.1. 智能数据合成 4
 - 5.2. RAG 和 Agent 场景支持 4
 - 5.3. 可配置的生成策略 4
 - 6. 框架集成与可观测性 4
 - 6.1. 主流框架集成 4
 - 6.2. 可观测性工具集成 4
 - 6.3. 统一的回调接口 4
- 二、 软件架构 5
- 三、 类的描述 6
 - 1. 核心模块 6
 - 2. 指标系统 7
 - 3. 测试集生成 8
 - 4. LLM 和嵌入 9
 - 5. 提示系统 10
 - 6. 优化器 10
 - 7. 后端存储 12
 - 8. 集成 13
 - 9. 工具模块 14
- 四、 类图 15
 - 1. 核心模块 15
 - 2. 指标系统 16
 - 3. 测试集生成 17
 - 4. LLM 和嵌入 18
 - 5. 提示系统 18
 - 6. 优化器 19

7. 后端存储	19
8. 集成	20
9. 工具模块	22
五、功能和类的对应关系	23
1. 管理评估样本数据	23
2. 评估指标体系	23
3. 模型评估执行	23
4. 实验管理与版本控制	24
5. 测试数据自动生成	24
6. 框架集成与可观测性	24
六、阅读收获	25
1. 亮点	25
1.1. 可插拔的后端存储架构	25
1.2. 实验驱动的评估框架	25
1.3. 异步优先的执行引擎	26
1.4. 双 Judge 评估策略	27
2. 反思	28
2.1. 缺乏统一的模型兼容层	28
2.2. 对 LangChain 框架的硬性依赖	29

一、项目功能描述

Ragas 作为一个开源的 Python 工具包，旨在将大语言模型（LLM）应用中传统的主观评审转变为系统化、可重复的评估 workflow。

其核心功能围绕三个维度展开：首先，Ragas 提供了一套结合 LLM 评测与传统指标的评估体系，专门用于量化 RAG 管道及 Agents 的输出质量；其次，Ragas 能够利用知识图谱与查询合成技术，从非结构化文档中自动生成合成测试数据集；最后，通过内置的适配器，Ragas 实现了与主流 LLM 供应商、嵌入模型及 LangChain、LlamaIndex 等开发框架的无缝集成。

1. 管理评估样本数据

1.1. 创建与管理评估数据

用户可以通过代码、.json 文件、.csv 文件、DataFrame 等多种来源导入评估数据，并组织成结构化的数据集，便于后续分析和评估。数据集支持多种存储后端，包括内存存储、本地文件系统、Google Drive 等。

1.2. 数据集导出与集成

数据集可以灵活导出为 .json、.csv、DataFrame 等格式，方便与其他数据分析工具和机器学习框架集成。支持批量操作和数据持久化，确保评估数据的可追溯性和可复用性。

2. 评估指标体系

2.1. 丰富的预置指标

Ragas 提供超过 30 种预置评估指标，覆盖 RAG 系统、AI Agent、多模态应用等多种场景。包括：

- **RAG 评估指标**：Faithfulness（忠实度）、Context Precision（上下文精确度）、Context Recall（上下文召回率）、Answer Relevance（答案相关性）
- **Agent 评估指标**：Tool Call Accuracy（工具调用准确度）、Goal Accuracy（目标达成度）
- **通用指标**：AspectCritic（方面评判）、Answer Correctness（答案正确性）、BLEU、ROUGE 等

2.2. 自定义指标能力

支持用户根据特定业务需求创建自定义评估指标。通过继承基础 Metric 类，用户可以定义自己的评分逻辑、提示词模板和评估标准，实现灵活的评估体系扩展。

3. 模型评估执行

3.1. 同步与异步评估

提供 evaluate() 和 aevaluate() 两种评估接口，支持同步和异步评估执行。可以批量评估大规模数据集，自动管理并发执行和资源分配，提高评估效率。

3.2. 多维度评估报告

自动生成详细的评估结果 (EvaluationResult)，包括每个样本的评分、统计信息、错误追踪等。支持进度显示、异常处理和结果持久化，帮助用户全面分析模型性能。

3.3. 灵活的 LLM 配置

支持多种 LLM 提供商（OpenAI、Anthropic、Azure、AWS Bedrock 等）和嵌入模型配置。通过 llm_factory 和 embedding_factory 实现统一的模型接口，方便切换和比较不同模型的评估效果。

4. 实验管理与版本控制

4.1. 实验追踪

通过 Experiment 模块系统化管理多次评估实验，追踪不同版本、不同参数配置下的模型表现。支持实验命名、结果对比和历史记录查询。

4.2. 版本控制集成

提供 `version_experiment()` 函数，与 Git 版本控制系统深度集成。自动创建实验分支、提交代码快照，确保实验的可复现性。每次实验都关联具体的代码版本，便于追溯和回滚。

4.3. 实验结果存储

实验结果可持久化到多种后端存储，支持长期保存和跨团队共享。通过统一的 Backend 接口，支持本地文件、云存储等多种存储方案。

5. 测试数据自动生成

5.1. 智能数据合成

使用 TestsetGenerator 自动生成高质量的测试数据集。基于知识图谱和 LLM 能力，生成覆盖多种场景的测试样本，包括简单问答、推理问题、多跳问题等。

5.2. RAG 和 Agent 场景支持

针对不同的应用场景提供专门的数据生成策略：

- **RAG 系统**：生成问题-上下文-答案三元组，覆盖不同难度级别
- **Agent 应用**：生成需要工具调用、多步推理的复杂测试场景

5.3. 可配置的生成策略

支持自定义数据生成参数，如问题类型分布、难度级别、数据量等。通过 Synthesizers 和 Transforms 模块，用户可以定制数据生成流程。

6. 框架集成与可观测性

6.1. 主流框架集成

与多个主流 LLM 应用开发框架无缝集成：

- **LangChain/LangGraph**：直接评估 Chain 和 Agent
- **LlamaIndex**：评估索引和查询引擎
- **Griptape、Swarm**：支持多 Agent 系统评估
- **R2R**：RAG 系统专项集成

6.2. 可观测性工具集成

与生产环境监控工具深度集成，构建完整的评估-监控-优化闭环：

- **LangSmith、Opik**：实时追踪评估过程
- **Helicone**：LLM 调用监控和成本分析
- **AG UI**：Agent 应用可视化调试

6.3. 统一的回调接口

提供标准化的回调机制，用户可以接入自定义的日志、监控和告警系统，实现评估过程的全面可观测。

二、软件架构

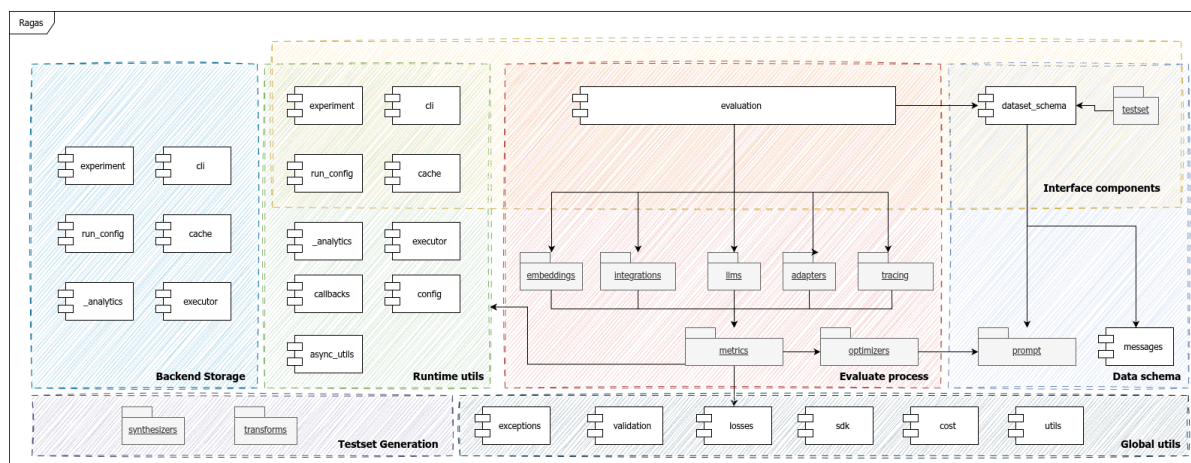


图1 Ragas 软件架构图

三、类的描述

1. 核心模块

包	模块	类	描述与作用
ragas	experiment	Experiment	实验环境管理器封装，继承自 DataTable，在迭代中进行版本控制与配置快照。
		ExperimentWrapper	实验函数的包装器类，将加上 @experiment 装饰器的测试函数打包为标准协议接口。
	executor	Executor	异步任务统一调度器，管理工作协程池、批处理限流与单样本防崩溃重启。
	run_config	RunConfig	全生命周期的执行统筹配置类，挂载超时、最大重试及并发线程数限制设定。
	dataset	DataTable	支持数据同步的基础表类，具备通用 CRUD 与底层将字典列转为对齐 Pandas DataFrame 的能力。
		Dataset	应用层的核心数据集容器，面向终端的实体载体，支持不同 Backend 的快照存储。
	dataset_schema	BaseSample	通用的 Pydantic 测例基类，约束测试输入的数据格式。
		SingleTurnSample	单轮测例数据类，含 user_input 甚至附带 retrieved_contexts，适用于基准 RAG 模型。
		MultiTurnSample	多轮对话测例数据类，搭载会话序列以追踪完整推演能力。
		EvaluationDataset	用于聚合特定验证集的类型安全池，为引擎打包分发异构单多轮样本提供保障。
		EvaluationResult	结构化的最终记分卡与反馈栈，包裹通过或失败评估的过程快照结果。
	messages	Message	消息基类，面向人类/机器的意图表达基础。
		Human/AIMessage	具体的聊天轮次对象，作为 MultiTurnSample 中的核心单元描摹会话时序。
		ToolCall	结构化表达外部函数和执行特征参数的模型，用于评估工具编排准确率。

2. 指标系统

包	模块	类	描述与作用
ragas	metrics.base	Metric	系统级顶层指标抽象，声明名字定义、同步计分及异步打分的实现协议规范。
		MetricWithLLM	基于提示词和模型生成的有状态指标基类，包裹 LLM 配置和多项重试挂载属性。
		SingleTurnMetric	适配于单轮事实或文档处理校验的特化基类，处理 SingleTurnSample 型对象。
		MultiTurnMetric	为智能体或代理设置的长路径、意图评估基类，处理 MultiTurnSample 数据结构流。
		SimpleBaseMetric	辅助指标混合类的泛型母类，是快速创建大语言模型辅助判别逻辑的基底。
		SimpleLLMMetric	快速构建模型主观验证任务的最简基类集，简化提示流开发。
		ModeMetric	处理统计和众数运算的数值抽象协议实体。
	metrics.validators	BaseValidator	对生成或推演结果的格式守门员接口。
		DiscreteValidator	针对类似 是/否、对/错 等分类任务判定的离散数据验证器。
		NumericValidator	处理比如相似度分数 (0.0-1.0) 这种含有连续精度概念量化产出的规范校验器。
	metrics.decorator	CustomMetric	装饰器内部返回的动态构建类工厂产物。
	metrics.discrete / numeric	DiscreteMetric	集成了基础大模型能力和固定类型验证器的混合型 (Mixin) 产物，专门输出离散属性。
		NumericMetric	集成了连续验证器 (NumericValidator) 和简单验证模块的具体打分特化指标子类。
	metrics._goal_accuracy	AgentGoalAccuracy WithReference	具备标准答案对照的代理意图成功率模型，基于 LLM 解析最终回复以核对既定事实流。
		AgentGoalAccuracy WithoutReference	零样本标准下的代理目标判断器，强依赖基于上下问进行自然语言语义判别的闭环内聚性。
	metrics._tool_call_accuracy	ToolCallAccuracy	专注校验多轮调用步骤下工具选择/函数填参的命中正确度的确定性计算指标系统类。

包	模块	类	描述与作用
	metrics._tool_call_f1	ToolCallF1	更进阶基于 Precision 与 Recall 结合，用以避免模型产生恶意“长尾胡乱工具调用尝试”的检测模型计算类。
	metrics.collections.base	BaseMetric	在集合模块为各个纯功能实现（如模糊匹配、字符串距离等）注入属性包装规范的基础。
		DistanceMeasure	基础字符串算法库用于表达匹配编辑尺度的枚举支持映射层。
		StringPresence	继承于大基类体系，专注于目标答案是否存在于生成的参考文档的验证规则实现子类。

3. 测试集生成

包	模块	类	描述
ragas	testset.graph	Node	知识图节点，封装实体属性与类型，是后续抽取、过滤和关系构建的基本单元。
		Relationship	知识图边，连接两个节点并记录关系类型与属性，用于多跳推理路径构造。
		KnowledgeGraph	测试集生成的核心数据结构，承载节点与关系，并提供增删改查与序列化能力。
	testset.persona	Persona	用户画像模型，描述角色名称与背景，用于生成风格化、多样化查询。
		PersonaGeneration Prompt	基于 PydanticPrompt 的人物生成提示模板，从知识图主题中自动构建 persona。
	testset.synthesizers.base	BaseScenario	场景抽象数据模型，统一约束 single-hop 和 multi-hop 的输入结构。
		BaseSynthesizer	查询合成器抽象基类，定义场景生成、样本生成与异步执行接口。
	testset.synthesizers.generate	TestsetGenerator	测试集生成入口类，协调文档转换、知识图构建、查询分布采样与最终 Testset 产出。
	testset.synthesizers.single_hop.base	SingleHopQuery Synthesizer	单跳查询合成器，针对单节点/单事实生成可直接检索的问题与答案。
	testset.synthesizers.multi_hop.base	MultiHopQuery Synthesizer	多跳查询合成器，组合多个概念或节点关系，生成需要跨段推理的问题。
	testset.synthesizers.testset_schema	TestsetSample	测试样本模型，保存 user_input、reference 等字段，作为评测最小单位。

包	模块	类	描述
	testset.transforms.base	Testset	测试集容器，聚合 TestsetSample 并提供导入导出与格式转换能力。
		BaseGraph Transformation	图转换抽象基类，统一抽取、分割、关系构建、过滤等步骤的执行协议。
		Extractor	从节点文本抽取结构化信息的基类，供 LLM、Regex、Embedding 提取器复用。
		RelationshipBuilder	关系构建器基类，按相似度或规则在节点间创建边。
	testset.transforms.extractors.llm_based	SummaryExtractor	LLM 摘要提取器，生成节点摘要，服务于主题归纳与后续关系构建。
	testset.transforms.relationship_builders.cosine	CosineSimilarity Builder	通过向量余弦相似度构建节点关系，形成语义邻接结构。

4. LLM 和嵌入

包	模块	类	描述与作用
ragas	llms.base	BaseRagasLLM	LLM 抽象基类，统一同步、异步生成、重试、温度策略与完成态检查。
		LangchainLLMWrapper	将 LangChain BaseLanguageModel 适配为 Ragas LLM 接口，兼容 n-completion 与回调链路。
		LlamaIndexLLMWrapper	将 LlamaIndex LLM 适配为 Ragas LLM 接口，复用统一的生成与追踪机制。
		InstructorBase RagasLLM	结构化输出 LLM 抽象层，面向 Pydantic 模型约束的 JSON 输出生成。
	llms.adapters.base	StructuredOutput Adapter	结构化输出适配器基类，屏蔽不同提供方在 schema 与调用方式上的差异。
	llms.adapters.litellm	LiteLLMAdapter	LiteLLM 结构化输出适配器，用统一接口接入多家模型服务。
	llms.litellm_llm	LiteLLMStructuredLLM	LiteLLM 的结构化 LLM 实现，直接提供 response_model 驱动的生成能力。
	embeddings.base	BaseRagasEmbedding	现代 embedding 抽象基类，定义 embed_text 与 embed_texts 的同步、异步语义。
		BaseRagasEmbeddings	LangChain Embeddings 兼容抽象层，支持 run_config、缓存和批处理行为统一。
		LangchainEmbeddings Wrapper	将 LangChain embeddings 包装为 Ragas 可用对象，便于旧代码迁移。

包	模块	类	描述与作用
	embeddings. openai_provider	OpenAIEmbeddings	OpenAI 向量模型实现，常用于语义检索和关系构建。
	embeddings. huggingface_provider	HuggingFace Embeddings	Hugging Face embedding 提供者实现，支持本地与远程模型。
	embeddings. litellm_provider	LiteLLMEmbeddings	LiteLLM embedding 统一接入层，便于跨提供方切换模型。

5. 提示系统

包	模块	类	描述与作用
ragas	prompt.base	BasePrompt	提示系统抽象基类，约束生成接口与序列化能力，是所有 prompt 组件的共同父类。
		StringPrompt	最简字符串提示模板，适用于轻量任务和快速格式化输出。
	prompt.mixin	PromptMixin	为合成器、提取器提供 prompt 注册、覆写和配置能力的混入类。
	prompt. pydantic_prompt	PydanticPrompt	核心泛型提示类，支持 Input、Output Pydantic 模型、JSON schema 约束与解析重试。
		RagasOutputParser	输出解析器，负责把 LLM 文本解析为目标 Pydantic 模型并统一异常语义。
		FixOutputFormat	输出修复提示模板，用于在解析失败时重构格式并提升可解析率。
	prompt. few_shot_ pydantic_prompt	ExampleStore	少样本示例仓库抽象接口，定义检索与更新示例的方法。
		FewShot PydanticPrompt	少样本增强版 PydanticPrompt，支持示例拼接与语义检索驱动示例选择。
	prompt. dynamic_few_shot	DynamicFewShotPrompt	动态 few-shot 提示类，按输入实时检索最相关示例以提升生成质量。
	prompt. multi_modal_prompt	ImageTextPrompt	多模态提示类，组合图片与文本上下文完成视觉和文本联合推理。
		ImageTextPromptValue	多模态提示值对象，封装可发送到底层模型的图文消息结构。
	prompt.metrics. base_prompt	BasePrompt	指标提示基类，专用于 metric 评分链路，支持多语言模板与结构化输出。

6. 优化器

包	模块	类	描述与作用
ragas	optimizers.base	Optimizer	抽象基类，定义 optimize 抽象方法及优化器通用属性。所有具体优化器应继

包	模块	类	描述与作用
			承此类并实现优化流程，保证接口一致性与可插拔性。
	optimizers.genetic	GeneticOptimizer	基于遗传算法的主要优化器实现，实现端到端的优化流程，并负责并行执行、进度汇报与最终候选选择。对 prompt 变异与候选评估有完整的控制逻辑。
		FormattedExamples	辅助数据模型，把原始示例转换为用于 Prompt 生成/反向工程的结构化示例列表。
		OutputInstruction	Pydantic 输出模型，包装生成的指令文本，作为 prompt 类输出的结构化格式。
		ReverseEngineer Prompt	Prompt 类，用于根据人工注释样本反推最可能的指示，作为生成初始候选的种子提示。
		ParentPrompts	Pydantic 输入模型，包装两个父提示字符串，作为交叉操作的输入结构。
		CrossOverPrompt	Prompt 类，用于对两个父提示执行“交叉”操作，生成子代提示，帮助在语义上结合父代信息以探索新候选。
	optimizers.genetic (feedback)	FeedbackExample	反馈示例数据模型，封装单个样本的输入/输出/期望输出，供反馈生成 prompt 使用。
		FeedbackMutation Input	反馈生成 Prompt 的输入模型，包含指令与若干 FeedbackExample，用于请求模型产生对指令的改进意见。
		FeedbackMutation Output	反馈生成 Prompt 的输出模型，包含若干具体反馈字符串，用于后续指令改写。
		FeedbackMutation Prompt	Prompt 类，生成针对给定指令与示例的改进反馈，限制反馈数量并避免建议通过修改示例来改进。
		FeedbackMutation PromptGeneration	Prompt 类，根据指令与反馈集合生成改写后的指令文本。
	optimizers.genetic (misc)	-	负责与 LLM 的交互、candidate 的生成与组合，为 GeneticOptimizer 的子步骤实现。
	optimizers.utils	hamming_distance	工具函数，计算候选预测向量之间的汉明距离，用于交叉变异阶段的父代选择。
	optimizers.dspy_optimizer	DSPyOptimizer	与外部库 DSPy 集成的高级优化器，将 Ragas 的 Prompt/数据/损失适配为 DSPy 格式，调用 MIPROv2 进行联合

包	模块	类	描述与作用
			示例与指令搜索，支持参数校验与结果缓存。适合需要更系统搜索能力的场景。
	optimizers. dspy_adapter	pydantic_prompt _to_dspy_signature, ragas_dataset _to_dspy_examples, create_dspy_metric, setup_dspy_llm	适配器集合，负责把 Ragas 的 PydanticPrompt、注释数据与 Loss 转换为 DSPy 所需的 Signature、Example 与 metric 函数，并配置 DSPy 使用的 LLM。
	optimizers. dspy_llm_wrapper	RagasDSPyLM	LLM 适配器类，将 BaseRagasLLM 包装为 DSPy 可调用的 LM 接口，处理同步/异步调用差异并返回文本生成结果，方便 DSPy 在优化过程中直接调用 Ragas 的 LLM 实现。

7. 后端存储

包	模块	类	描述与作用
ragas	backends.base	BaseBackend	抽象后端基类，定义存取数据集与实验的统一接口契约，要求子类实现持久化细节并在缺失时抛出 FileNotFoundError，确保返回格式为 List[Dict]。
	backends.inmemory	InMemoryBackend	内存后端实现，将数据集与实验保存在进程内字典中，适用于测试与临时计算，提供深拷贝保护以避免外部修改，并实现与 BaseBackend 相同的公共接口。
	backends.local_csv	LocalCSVBackend	本地 CSV 文件后端，按目录分离 datasets/ 与 experiments/。
	backends.local_jsonl	LocalJSONLBackend	本地 JSONL 文件后端，以 JSONL 保存数据，保留嵌套结构与常见 Python 类型，适用于复杂数据与大规模流式写入场景。
	backends.registry	BackendRegistry	后端注册表类，单例模式管理可用后端的发现、注册、别名与创建，支持通过 setuptools entry points 自动发现插件后端并提供 dict-like 访问接口。
	backends. gdrive_backend	GDriveBackend	Google Drive 后端实现，将数据集与实验保存为 Google Sheets，支持 OAuth 与 Service Account 两种认证方式，负责在指定 Drive 文件夹下创建/读取表格并做类型转换与错误处理。
	backends.utils	MemorableNames	实用类，生成可记忆且通常唯一的名称，用于为实验或数据集生成人类友好

包	模块	类	描述与作用
			的默认标识， 内置去重与批量生成方法。

8. 集成

包	模块	类	描述与作用
ragas.	integrations.tracing.langfuse	LangfuseTrace	Langfuse trace 的封装类，包装 Langfuse 返回的 trace 对象，提供统一的 get_url() 与简单过滤方法，并在缺少依赖时通过类型桩保持兼容。
		TraceWithFullDetails	描述 Langfuse trace 的详细数据结构类型，包含完整的 trace 元数据、观测、分数与其他辅助字段，便于消费端解析与显示。
		Observation	Langfuse trace 中单个观测项的类型桩，表示在 trace 中记录的单条观测记录的字段与结构，供代码在缺少真实依赖时类型检查使用。
	integrations.tracing.mlflow	MLflowTrace	MLflow trace 的封装类，将 mlflow 的 trace 对象包装为统一接口，负责根据环境构造可访问的 trace URL 并提供按 span 名称过滤的能力。
		Span	mlflow 中单个 span 的类型/兼容桩，表示 trace 中的 span 条目。
		Trace	mlflow trace 的类型/兼容桩，包含 trace 的元信息，并提供按名称搜索 spans 的方法，供封装类构造 URL 与做过滤。
	integrations.langchain	EvaluatorChain	将 Ragas 的 Metric 适配为 LangChain/ LangSmith 的 Chain/ RunEvaluator，负责输入/输出键映射、将 LLM/Embeddings 懒注入为 LangChain 可用 wrapper，并在 chain 调用时执行 metric 评估。
	integrations.opik	OpikTracer	继承并扩展 Opik/LangChain 的 tracer，拦截评估流程的 start/end 事件，区分评估 run 与子 trace，并向 Opik 平台上报评分与元数据。
	integrations.ag_ui	AGUIEventCollector	AG-UI 协议事件收集与重构核心类，从流式事件或快照重建完整消息，管理 streaming triads 与工具调用状态，最终输出可供 Ragas 评估使用的消息列表。

包	模块	类	描述与作用
	integrations.helicone	HeliconeSingleton	Helicone 集成的单例配置类，集中管理 API 密钥、base_url 与可选缓存配置，提供 default_headers() 生成用于 Helicone 请求的 header 集合并暴露是否启用的判断。
		CacheConfig	缓存配置数据类，定义缓存的 TTL 与最大大小，用于 Helicone 客户端的缓存策略配置。

9. 工具模块

包	模块	类	描述与作用
ragas	utils	DeprecationHelper	处理过时类/函数的包装器，在调用被弃用的 API 时发出 DeprecationWarning 并委托到新的目标实现；当通过属性访问时也会发出警告，便于平滑迁移和兼容旧导出。
		_ContextualFormatter	自定义日志格式化器，扩展 logging.Formatter，在日志记录中注入 UTC/local 时间、Ragas 用户 ID、应用版本等上下文字段，统一日志格式以便可观测性与追踪。
		ProgressBarManager	进度条管理器，封装 tqdm 的单条与嵌套批处理进度条创建与更新逻辑，便于在批量/异步任务中展示进度与分批进度细节。
		MemorableNames	可记忆名称生成器，生成类似 adjective_scientist 的友好名称并维护去重集合，支持单个/批量生成，常用于为实验或数据集生成默认标识符。

四、类图

1. 核心模块

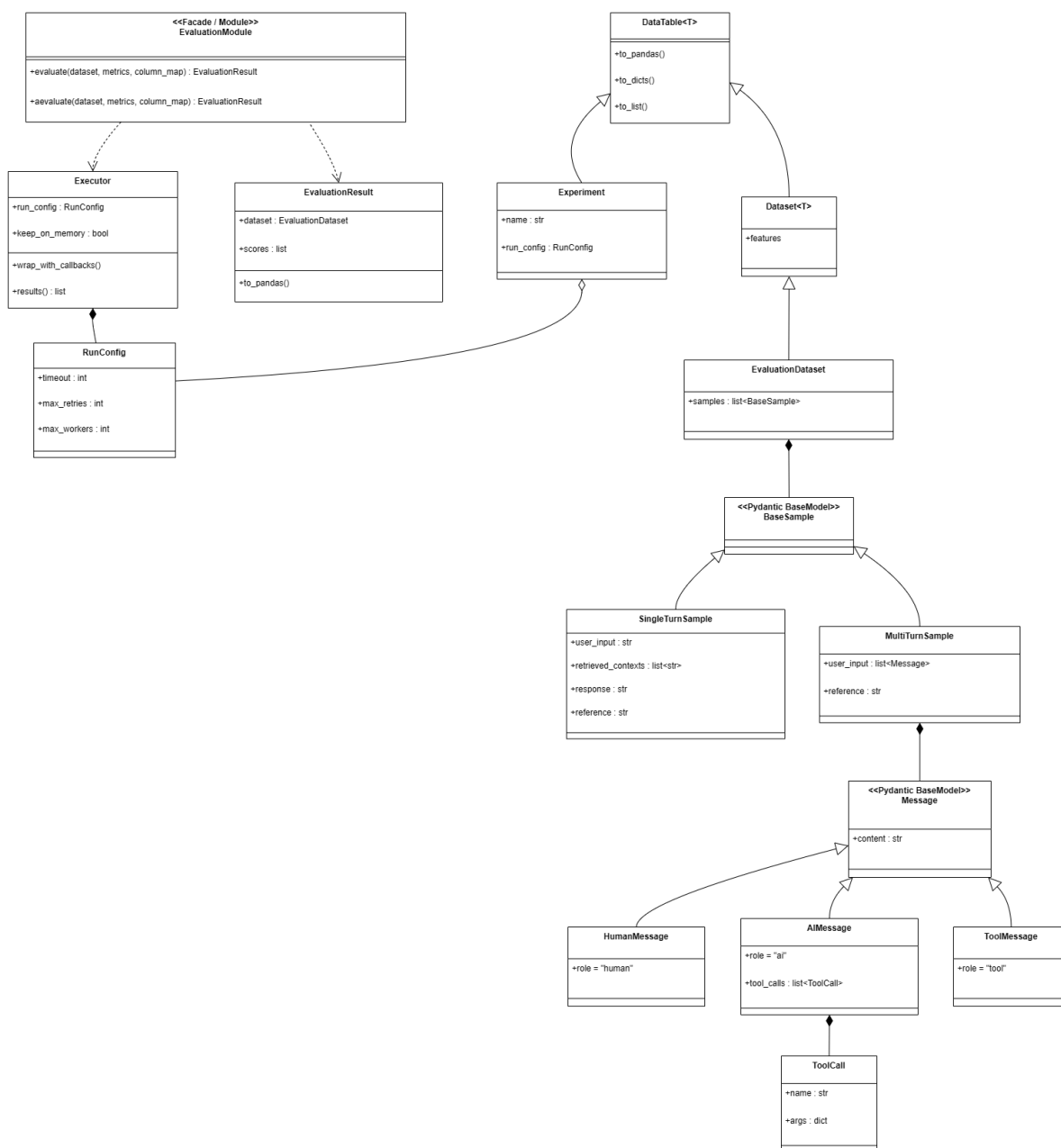


图 2 核心模块类图

2. 指标系统

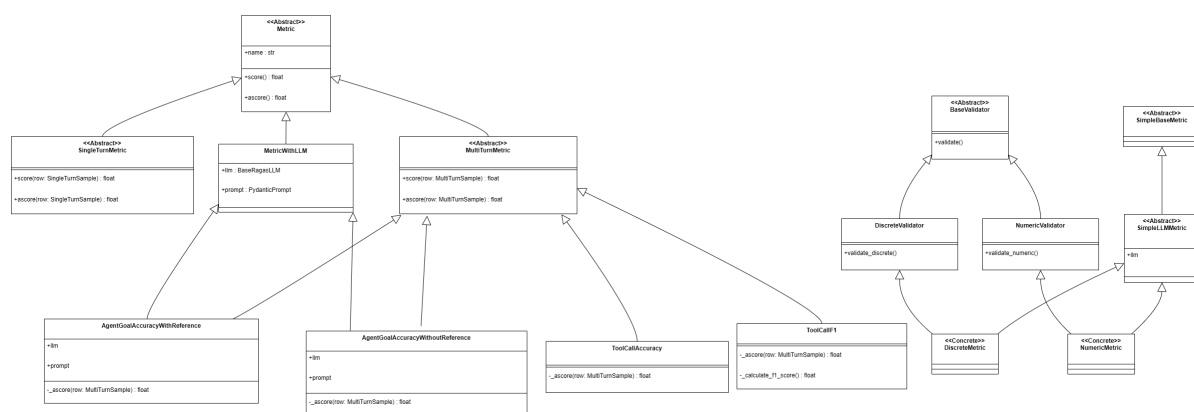
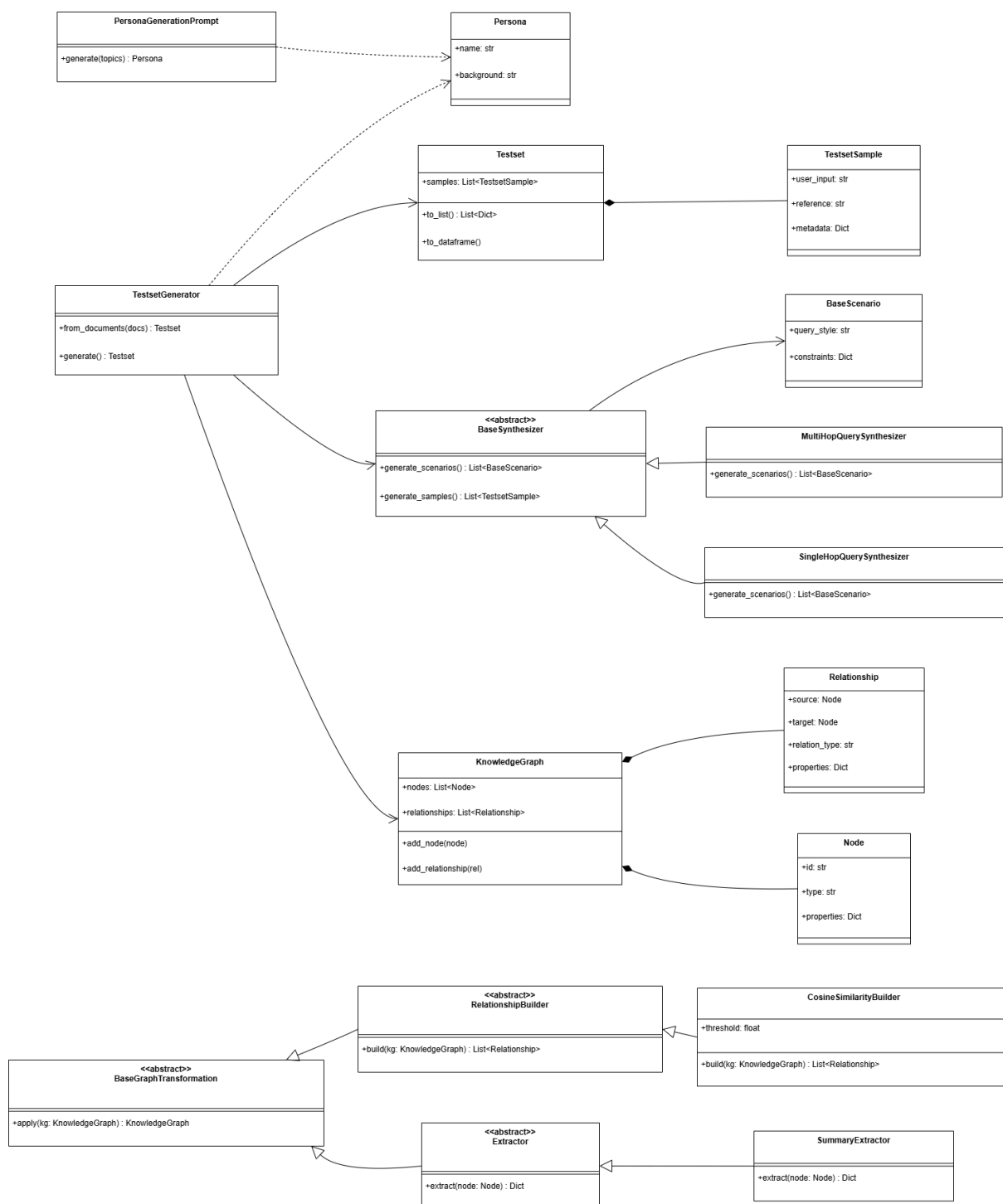


图3 指标系统类图

3. 测试集生成



4. LLM 和嵌入

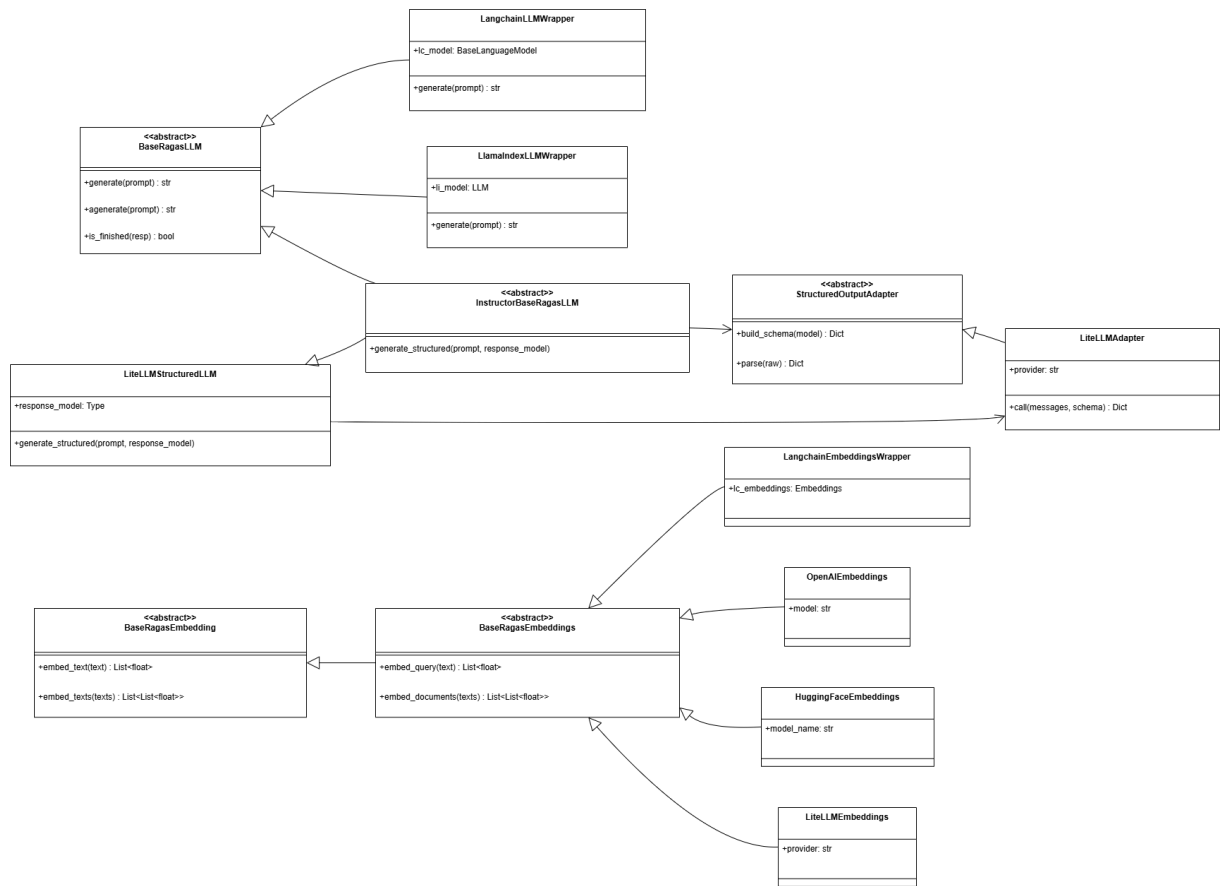


图 5 LLM 和嵌入类图

5. 提示系统

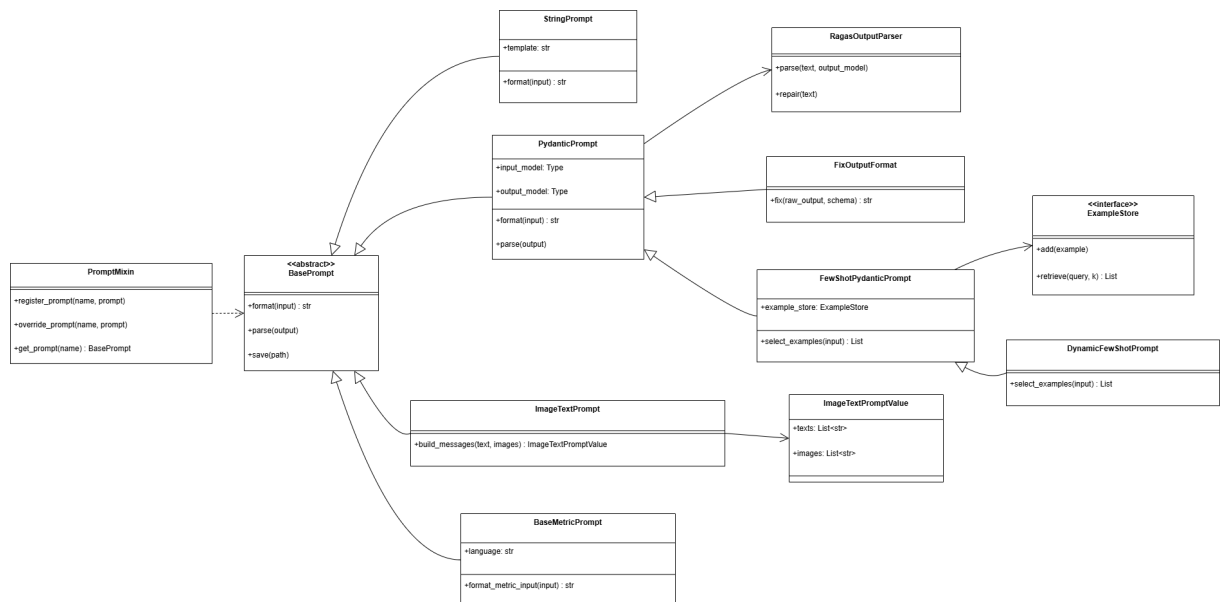


图 6 提示系统类图

6. 优化器

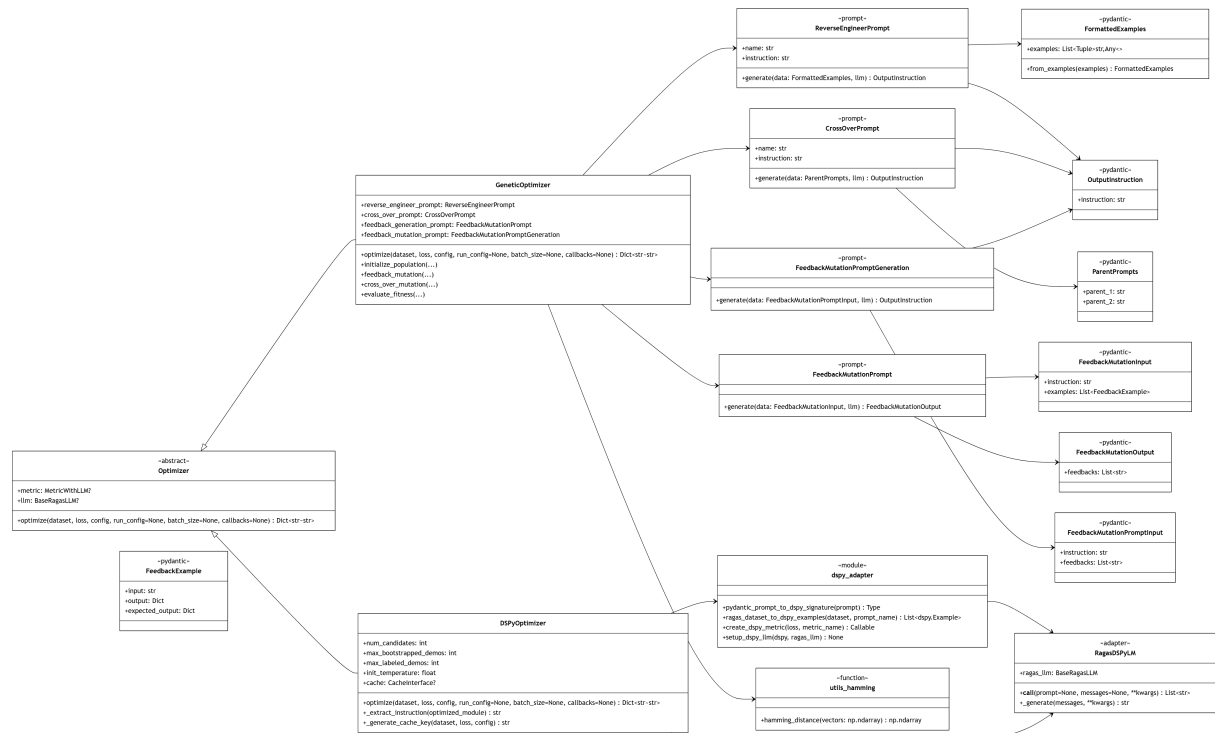


图 7 优化器类图

7. 后端存储

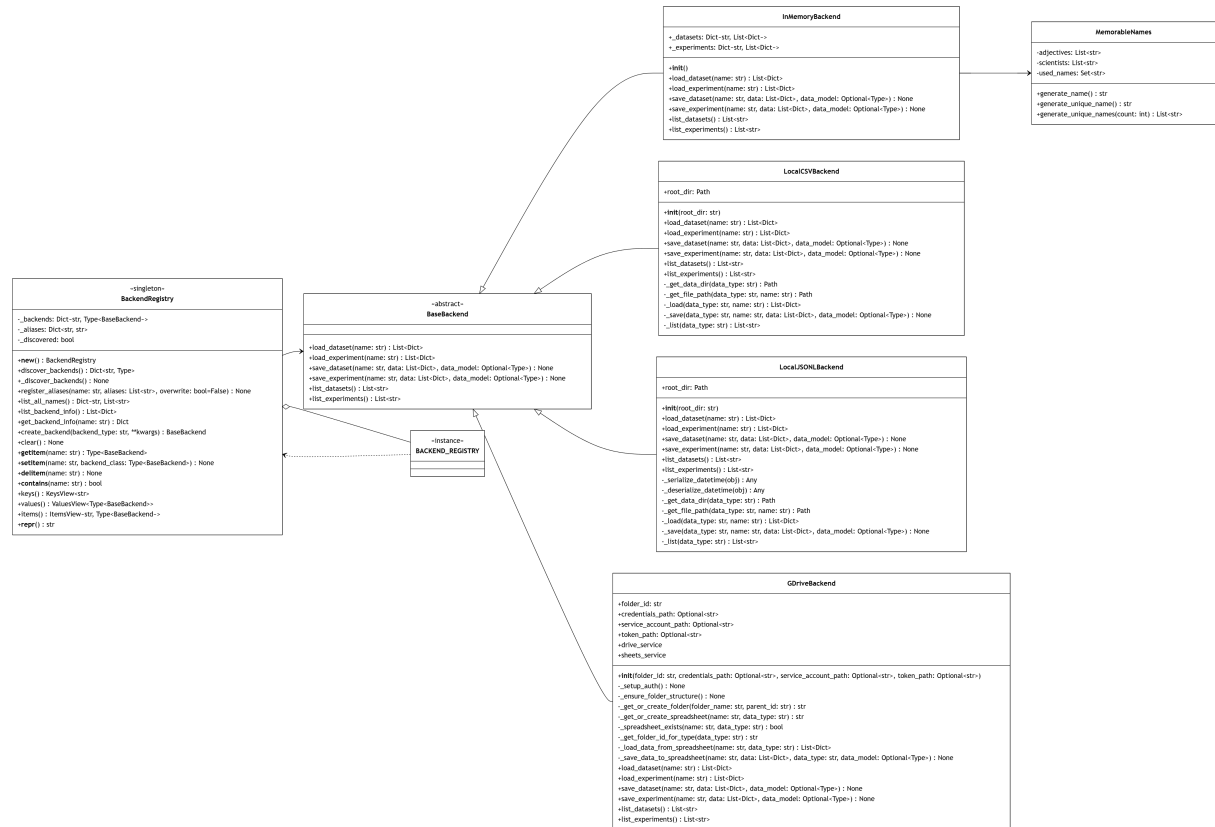


图 8 后端存储类图

8. 集成

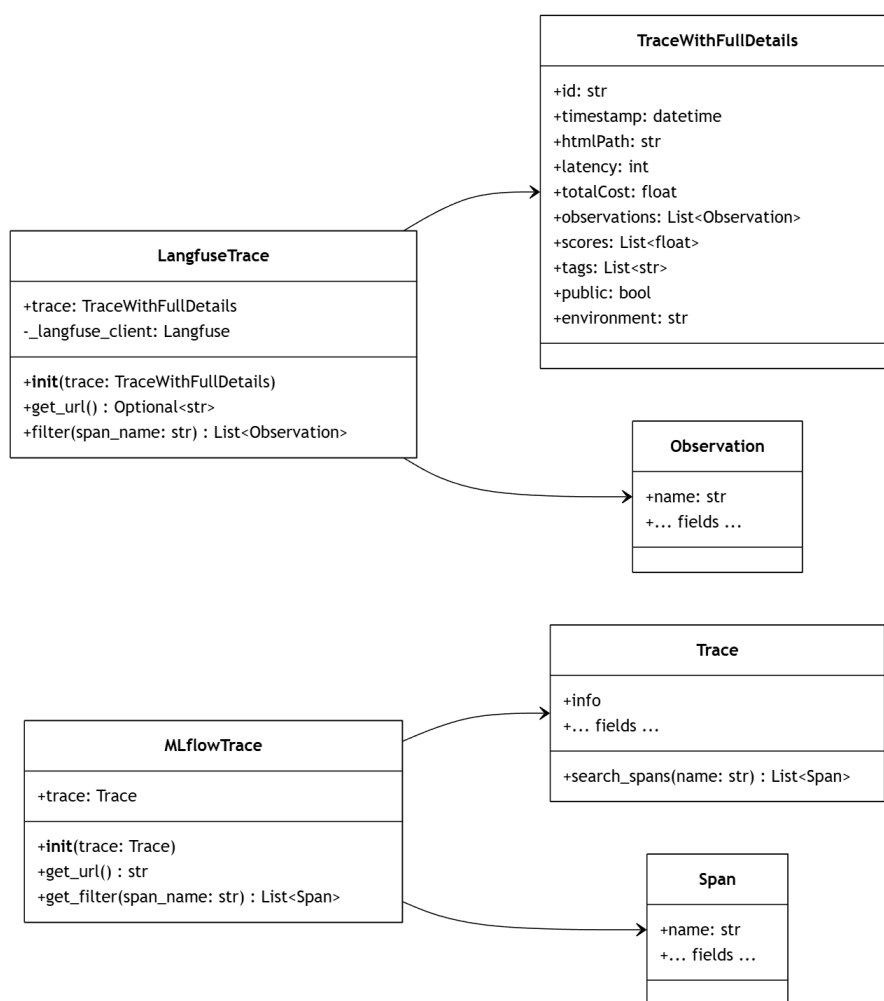


图9 集成类图 1

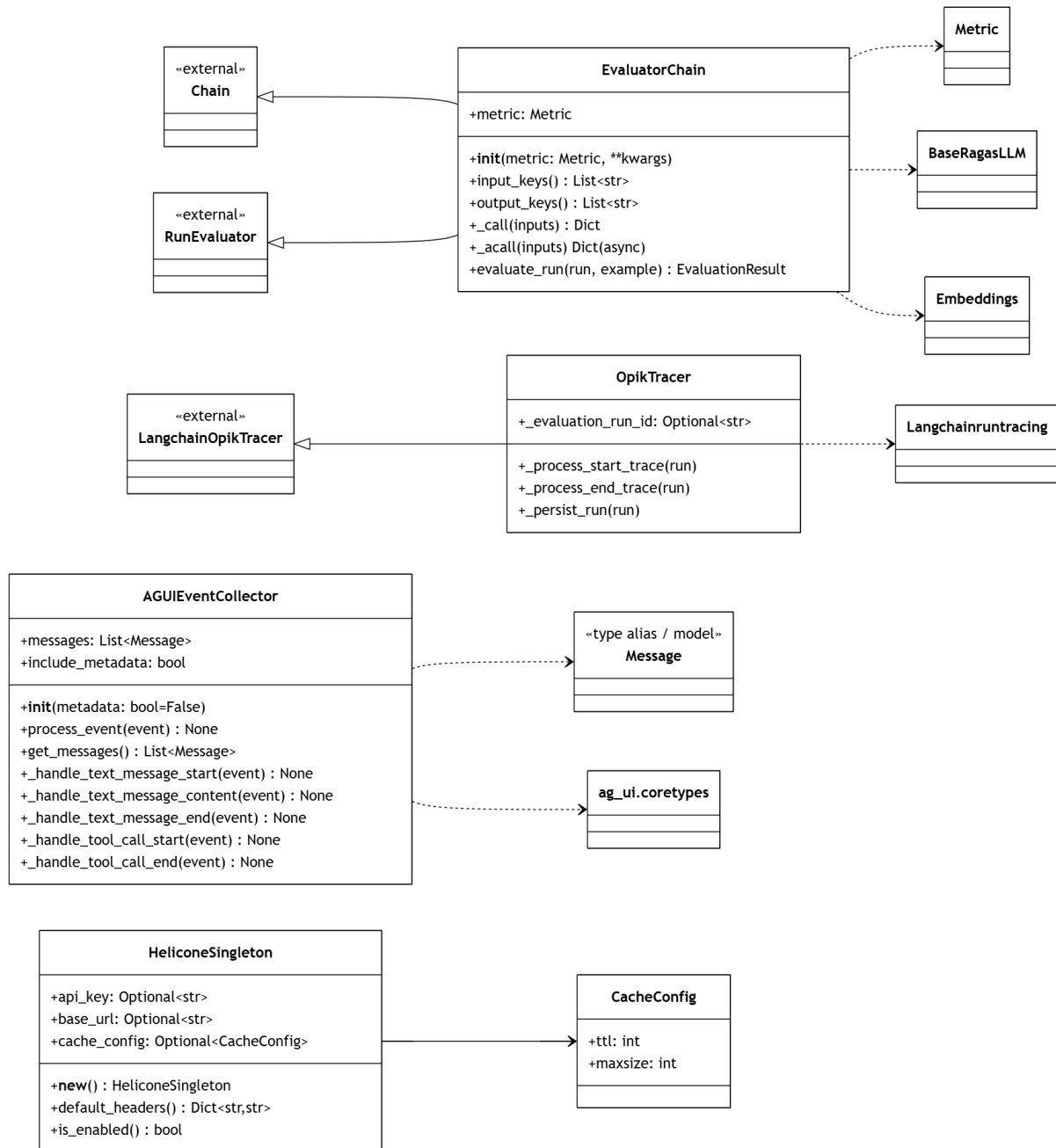


图 10 集成类图 2

9. 工具模块

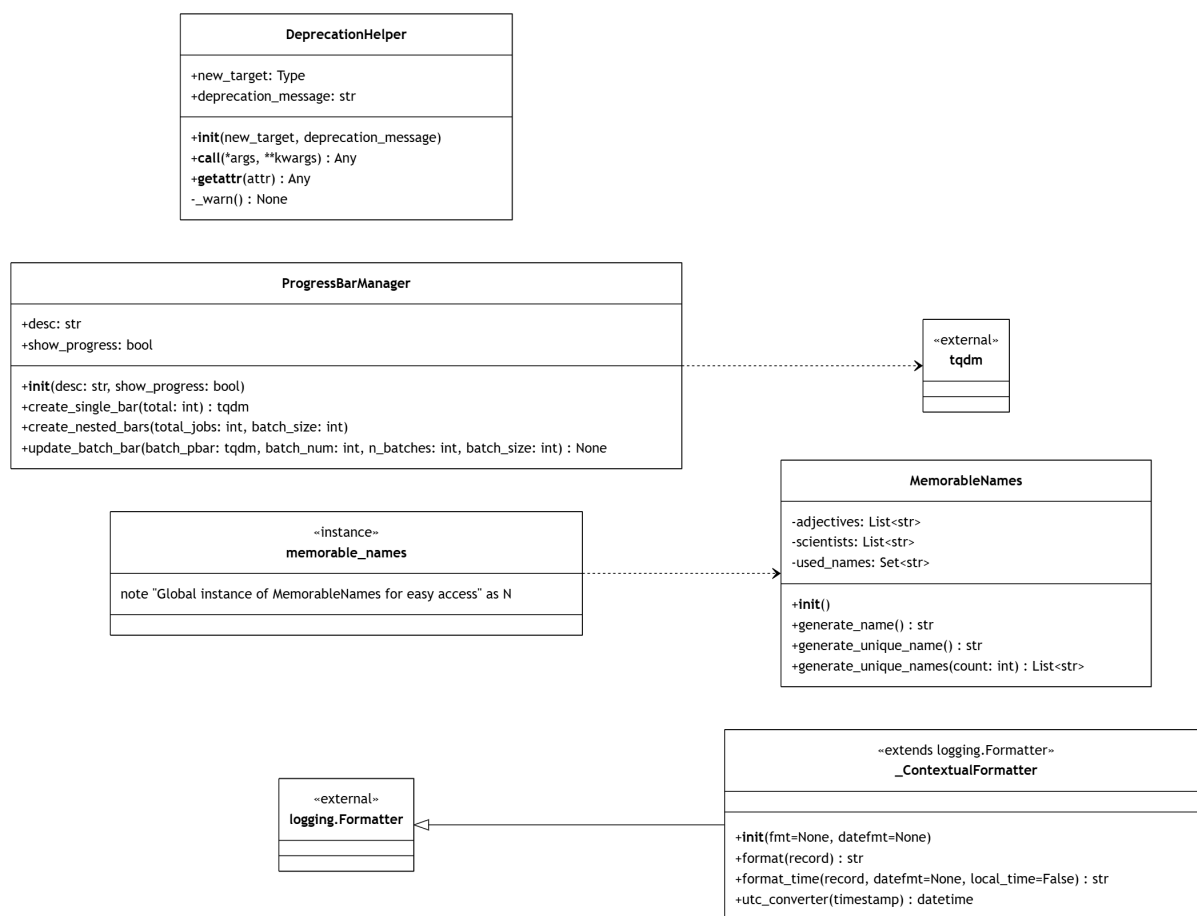


图 11 工具模块类图

五、功能和类的对应关系

1. 管理评估样本数据

用户功能	系统功能	实现模块	实现方法/类
管理评估样本数据	创建与管理数据	dataset	Dataset
			DataTable
		dataset_schema	SingleTurnSample
			MultiTurnSample
	数据集聚合与分发	dataset_schema	EvaluationDataset
			BaseSample
			EvaluationResult
	上下文管理	messages	Message

2. 评估指标体系

用户功能	系统功能	实现模块	实现方法/类
评估指标体系	丰富的预置指标	metrics.base	Metric
			SingleTurnMetric
			MultiTurnMetric
			MetricWithLLM
		metrics.discrete	DiscreteMetric
	Agent 专项评估	metrics._tool_call_accuracy	ToolCallAccuracy
		metrics._tool_call_f1	ToolCallF1
		metrics._goal_accuracy	AgentGoalAccuracyWithReference
			AgentGoalAccuracyWithoutReference
	自定义指标能力	metrics.base	SimpleBaseMetric
		metrics.decorator	CustomMetric
		metrics.validators	BaseValidator

3. 模型评估执行

用户功能	系统功能	实现模块	实现方法/类
模型评估执行	同步与异步评估	executor	Executor
		run_config	RunConfig
		metrics.base	Metric
	多维度评估报告	dataset_schema	EvaluationResult
			EvaluationDataset
	灵活的 LLM 配置	llms.base	BaseRagasLLM
			LangchainLLMWrapper

4. 实验管理与版本控制

用户功能	系统功能	实现模块	实现方法/类
实验管理与版本控制	实验追踪	experiment	Experiment
			ExperimentWrapper
	版本控制集成	experiment	version_experiment()
	实验结果存储	backends	BaseBackend
			LocalCSVBackend
			LocalJSONLBackend
			GDriveBackend

5. 测试数据自动生成

用户功能	系统功能	实现模块	实现方法/类
测试数据自动生成	智能数据合成	testset.synthesizers.generate	TestsetGenerator
		testset.graph	KnowledgeGraph
		testset.synthesizers.base	BaseSynthesizer
	RAG 和 Agent 场景支持	testset.synthesizers.single_hop.base	SingleHopQuerySynthesizer
		testset.synthesizers.multi_hop.base	MultiHopQuerySynthesizer
	可配置的生成策略	testset.transforms.base	BaseGraphTransformation
			Extractor
			RelationshipBuilder

6. 框架集成与可观测性

用户功能	系统功能	实现模块	实现方法/类
框架集成与可观测性	主流框架集成	integrations.langchain	EvaluatorChain
		integrations.ag_ui	AGUIEventCollector
		llms.base	LangchainLLMWrapper
			LlamaIndexLLMWrapper
	可观测性工具集成	integrations.tracing.langfuse	LangfuseTrace
		integrations.tracing.mlflow	MLflowTrace
		integrations.opik	OpikTracer
		integrations.helicone	HeliconeSingleton
	统一的回调接口	callbacks	Callbacks

六、阅读收获

1. 亮点

在阅读 Ragas 框架的代码实现过程中，我们发现了很多值得借鉴的架构设计和具体实现的方法。

1.1. 可插拔的后端存储架构

Ragas 框架采用了高度模块化的后端存储设计，通过定义统一的抽象基类 `BaseBackend`，将数据持久化逻辑与核心评估业务逻辑彻底解耦。

这种设计不仅提供了内存、本地文件（CSV/JSONL）以及云端存储（Google Drive）等多种开箱即用的实现方案，以适应从快速原型开发到团队协作的不同场景，还利用 Python 的 `entry points` 机制构建了灵活的插件注册系统，使得开发者能够在不侵入核心代码库的前提下，轻松扩展自定义存储后端，从而极大地提升了系统的可维护性和对不同基础设施的适应能力。

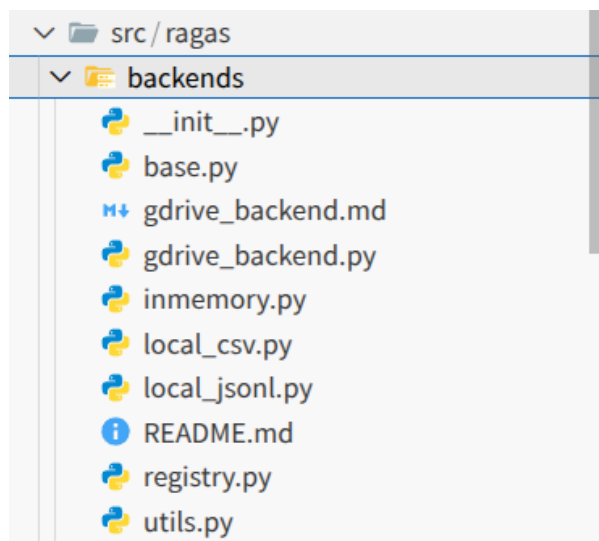


图 12 后端的文件结构

```
1 # pyproject.toml
2 [project.entry-points."ragas.backends"]
3 "local/csv" = "ragas.backends.local_csv:LocalCSVBackend"
4 "local/jsonl" = "ragas.backends.local_jsonl:LocalJSONLBackend"
5 "inmemory" = "ragas.backends.inmemory:InMemoryBackend"
6 "gdrive" = "ragas.backends.gdrive_backend:GDriveBackend"
```

1.2. 实验驱动的评估框架

Ragas 创新性地引入了实验驱动的评估范式，通过**装饰器模式**将普通的评估函数无缝转换为可追踪、可管理的实验对象，实现了评估逻辑与实验元数据的有机结合。

使用装饰器 `@experiment`，开发者可以轻松定义评估函数的输入输出格式、指定后端存储方案，并自动获得版本控制和结果追踪等功能，从而极大地简化了评估流程的管理和结果的可复现性。这种可以提升实验效率和结果可靠性的设计是值得我们在未来的项目中借鉴的。

```
1 # /examples/ragas_examples/ag_ui_agent_experiments/experiments.py
2 @experiment(
3     experiment_model=CustomModel,
4     backend="local/csv",
5     name_prefix="rag_v1"
6 )
```

```

7  async def evaluate_rag(row):
8      # 评估逻辑
9      response = await rag_system(row["query"])
10     return {"response": response, **row}
11
12 # 自动版本化
13 version_experiment("rag_v1", create_branch=True)

```

1.3. 异步优先的执行引擎

在执行效率方面，Ragas 构建了一个高效的异步优先执行引擎，通过 `Executor` 类对 Python 的 `asyncio` 库进行了深度封装，实现了评估任务的高并发处理。

该引擎不仅支持细粒度的任务并发以显著缩短大规模数据集的评估时间，还引入了灵活的批处理机制来平衡内存消耗与吞吐量，同时完善的任务取消机制和异常处理策略保证了在长时间运行任务中的系统稳定性与控制力。

在后续我们复现 Agent 评估框架的过程中，面对大批量的评估任务时，可以借鉴 Ragas 的这种异步执行设计，以提升评估效率和系统的健壮性。

```

1  @dataclass
2  class Executor:
3      async def _process_batched_jobs(
4          self, jobs_to_process, progress_manager, max_workers, results
5      ):
6          """Process jobs in batches with nested progress tracking."""
7          batch_size = self.batch_size or len(jobs_to_process)
8          batches = batched(jobs_to_process, batch_size)
9          overall_pbar, batch_pbar, n_batches = progress_manager.create_nested_bars(
10              len(jobs_to_process), batch_size
11          )
12
13          with overall_pbar, batch_pbar:
14              for i, batch in enumerate(batches, 1):
15                  # 检查任务取消状态，及时退出以节省资源
16                  if self.is_cancelled():
17                      break
18
19                  progress_manager.update_batch_bar(batch_pbar, i, n_batches,
20                      len(batch))
21
22                  # 创建协程列表以并发执行当前批次的所有任务
23                  coroutines = [
24                      afunc(*args, **kwargs) for afunc, args, kwargs, _ in batch
25                  ]
26
27                  async for result in process_futures(
28                      as_completed(
29                          coroutines, max_workers, cancel_check=self.is_cancelled
30                      )
31                  ):
32                      # 如果任务执行过程中出现异常，根据配置决定是否抛出异常或继续处理剩余任务
33                      if isinstance(result, Exception) and self.raise_exceptions:
34                          raise result
35                      results.append(result)
36                      batch_pbar.update(1)
37                  # 更新总体进度条，反映已处理的任务数量

```

37

`overall_pbar.update(len(batch))`

1.4. 双 Judge 评估策略

在提升基于 LLM 的评估 (LLM-as-a-Judge) 的客观性方面, Ragas 引入了创新的双 Judge 评估策略, 借鉴了 NVIDIA 的相关研究成果, 通过引入视角的对称性来抵消单一模型可能存在的评估偏差。

该策略设计了互为镜像的两个评估角色分别对“用户回答-参考答案”对进行独立打分, 并将异构的评分结果标准化到统一区间后进行融合, 这种方法不仅在不依赖大量人工标注数据的情况下显著提升了评估结果的一致性与鲁棒性, 还通过算法设计层面的优化有效降低了模型幻觉对评估质量的影响。这种评估方法在我们未来设计 Agent 评估框架时, 可以作为借鉴对象, 以提升评估的客观性和可靠性。

```
1 class AnswerAccuracy(BaseMetric):
2     """
3     Answer Accuracy metric using dual-judge evaluation.
4
5     Measures answer accuracy compared to ground truth using a dual-judge system.
6     This metric averages two distinct judge prompts to ensure robust evaluation.
7
8     The metric uses NVIDIA's proven dual-judge approach:
9     1. Judge 1: Direct User Answer vs Reference Answer comparison
10    2. Judge 2: Swapped perspective for fairness
11    3. Average both judges for final score
12
13    Rating scale: 0 (no match), 2 (partial match), 4 (exact match)
14    Final score: Average of both judges converted to 0.0-1.0 scale
15
16    Attributes:
17        llm: Modern instructor-based LLM for dual-judge evaluation
18        name: The metric name
19        allowed_values: Score range (0.0 to 1.0, higher is better)
20        max_retries: Maximum retry attempts for invalid ratings
21    """
22
23    async def ascore(
24        self, user_input: str, response: str, reference: str
25    ) → MetricResult:
26        """
27        Calculate answer accuracy score using dual-judge evaluation.
28
29        Args:
30            user_input: The original question
31            response: The user's answer to evaluate
32            reference: The ground truth reference answer
33
34        Returns:
35            MetricResult with answer accuracy score (0.0-1.0, higher is better)
36        """
37        # Input validation
38        if not user_input:
39            raise ValueError(
40                "user_input is missing. Please add user_input to the test sample."
41            )
42        if not response:
```

```
43         raise ValueError(  
44             "response is missing. Please add response to the test sample."  
45         )  
46     if not reference:  
47         raise ValueError(  
48             "reference is missing. Please add reference to the test sample."  
49         )  
50  
51     # Get ratings from both judges  
52     judge1_rating = await self._get_judge_rating(  
53         self.judge1_prompt, user_input, response, reference  
54     )  
55     judge2_rating = await self._get_judge_rating(  
56         self.judge2_prompt, user_input, reference, response  
57     ) # Note: swapped order for judge 2  
58  
59     # Average the scores (convert from 0,2,4 scale to 0.0-1.0)  
60     score = self._average_scores(judge1_rating / 4.0, judge2_rating / 4.0)  
61  
62     return MetricResult(value=float(score))
```

2. 反思

在深入阅读 Ragas 庞大而复杂的代码库过程中，我们也发现了一些值得改进的地方。

2.1. 缺乏统一的模型兼容层

当前实现的 `_map_openai_params` 方法存在大量针对特定未来模型版本的硬编码，在架构设计上存在明显的局限性。通过在代码中直接嵌入针对特定版本号（如 o1 至 o9 以及 gpt-5 至 gpt-19）的正则化判断与逻辑约束，系统严重违反了软件工程中的“开闭原则”。这种设计意味着框架对扩展是不开放的，每当模型供应商发布超出预设命名范围的新模型时，开发者必须介入并修改核心源代码，而非通过简单的配置更新来适配新特性。

```
1 def _map_openai_params(self) → t.Dict[str, t.Any]:  
2     """Map parameters for OpenAI/Azure reasoning models with special  
3     constraints.  
4  
5     Pattern-based matching for future-proof coverage:  
6     - 0-series: o1, o2, o3, o4, o5, ... (all reasoning versions)  
7     - GPT-5 series: gpt-5, gpt-5-*, gpt-6, gpt-7, ... (all GPT-5+ models)  
8     - Other: codex-mini  
9     """  
10  
11     mapped_args = self.model_args.copy()  
12  
13     model_lower = self.model.lower()  
14  
15     # Pattern-based detection for reasoning models that require  
16     max_completion_tokens  
17     # Uses prefix matching to cover current and future model variants  
18     def is_reasoning_model(model_str: str) → bool:  
19         """Check if model is a reasoning model requiring  
20         max_completion_tokens."""  
21         # 0-series reasoning models (o1, o1-mini, o1-2024-12-17, o2, o3, o4,  
22         o5, o6, o7, o8, o9)  
23         # Pattern: "o" followed by single digit 1-9, then optional "-" or end  
24         of string  
25         # TODO: Update to support o10+ when OpenAI releases models beyond o9
```

```

20         if (
21             len(model_str) >= 2
22             and model_str[0] == "o"
23             and model_str[1] in "123456789"
24         ):
25             # Allow single digit o-series: o1, o2, ..., o9
26             if len(model_str) == 2 or model_str[2] in ("-", "_"):
27                 return True
28
29             # GPT-5 and newer generation models (gpt-5, gpt-5-*, gpt-6, gpt-7, ...,
gpt-19)
30             # Pattern: "gpt-" followed by single or double digit >= 5, max 19
31             # TODO: Update to support gpt-20+ when OpenAI releases models beyond
gpt-19
32             if model_str.startswith("gpt-"):
33                 version_str = (
34                     model_str[4:].split("-")[0].split("_")[0]
35                 ) # Get version number
36                 try:
37                     version = int(version_str)
38                     if 5 <= version <= 19:
39                         return True
40                 except ValueError:
41                     pass
42
43             # Other specific reasoning models
44             if model_str == "codex-mini":
45                 return True
46
47             return False
48
49             # 省略其他模型参数映射逻辑 ...
50
51         return mapped_args

```

2.2. 对 LangChain 框架的硬性依赖

Ragas 在架构设计上过度耦合了 LangChain 生态系统，对于整个软件的可扩展性和生态兼容性造成了严重的限制。

这种硬性依赖不仅体现在核心依赖库（如 langchain-core 和 langchain-community）的直接引入，更深层地渗透到了其内部的数据交换协议中。由于框架底层强行绑定了 LangChain 特有的 LLMResult 数据结构（如特定的 generations 多维数组格式），导致其在接入非 OpenAI 或非 LangChain 原生模型时表现出极差的兼容性。

在当前 LLM 推理后端（如 vLLM, TensorRT-LLM）快速演进的背景下，这种缺乏抽象层和标准化接口规范现状，使得框架难以原生支持高吞吐的工业级评价场景，成为制约大规模评估任务性能提升的瓶颈。

在 [Issue #2058](#) 中，由于这种缺乏接口隔离的设计，迫使开发者在尝试使用 vLLM 等高性能推理引擎加速评估时，必须在外部编写冗余且别扭的异步转换代码，仅仅是为了对齐框架内部的封装逻辑，这种无意义的格式转换极大地降低了系统集成的灵活性与执行效率。

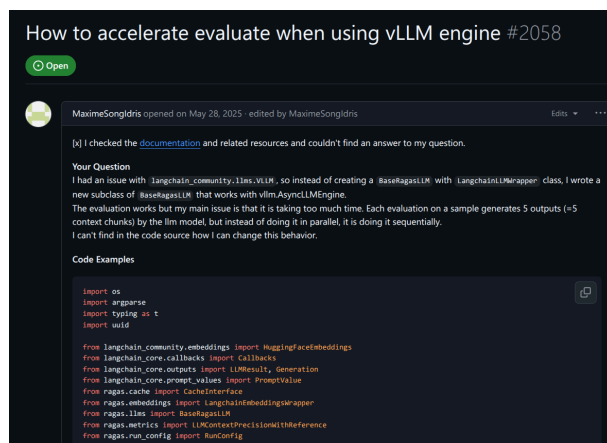


图 13 Issue #2058 中的 vLLM 适配问题